


深圳大学 计算机与软件学院

College of Computer Science and Software Engineering of Shenzhen University

系统编程

基于TaiShan服务器/openEuler OS 的实践

第三讲：多线程编程 – 线程控制

实验一学生实验报告评析

1. 有一个全局变量i初值为5，父进程对其进行加1操作，子进程看到的i的取值将会是多少？（10分）

1. /home/mobaxterm | 2. 172.31.234.200 (wujunyi_201719

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>


int main(int argc, char *argv[])
{
    int i = 5;

    pid_t pid = fork();

    for(int k=0;k<3;k++){
    if (pid != 0) {
        printf("Parent has i = %d\n", ++i);
    }
    else {
        printf("Chile has i = %d\n", i);
    }
    }

    printf("Bye from process %d with i = %d\n", getpid(), i);

    return 0;
}
```

```
[wujunyi_2017192040@taishan02-vm-4 exp1]$ ./Q3
Parent has i = 6
Parent has i = 7
Parent has i = 8
Bye from process 111967 with i = 8
Chile has i = 5
Chile has i = 5
Chile has i = 5
Bye from process 111968 with i = 5
[wujunyi_2017192040@taishan02-vm-4 exp1]$
```

可见，父进程的 i 循环自增三次，最终输出 8，而子进程的 i 始终为 5，最终输出为 5。
进一步，查看父子进程中变量 i 的地址，如下所示：

```
[wujunyi_2017192040@taishan02-vm-4 exp1]$ ./Q3
Parent has i = 6
Chile has i = 5
Parent has i = 7
Chile has i = 5
Parent has i = 8
Chile has i = 5
Bye from process 112088 with i = 8, Address:-624949020
Bye from process 112089 with i = 5, Address:-624949020
[wujunyi_2017192040@taishan02-vm-4 exp1]$
```

由图 63，父子进程输出的 i 变量的地址却是相同的，然而二者 i 的数值却不相同。事实上，在 Linux 操作系统中，程序打印输出的变量地址通常为**虚拟内存地址**，通过内核映射到实际的物理内存。i 变量的值是独立的，即父子进程中 i 的物理地址是不一样的，所以物理内存中 i 变量应该为两个，两者都映射到了同一个虚拟内存地址。

信而有征

Talk is cheap, show me the code …

2. 172.31.234.200 (szu) | 3. 172.31.234.200 (szu) (1)

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdint.h>

void mem_addr(unsigned long vaddr, unsigned long *paddr)
{
    int pageSize = getpagesize();

    unsigned long v_pageIndex = vaddr / pageSize;
    unsigned long v_offset = v_pageIndex * sizeof(uint64_t);
    unsigned long page_offset = vaddr % pageSize;
    uint64_t item = 0;

    int fd = open("/proc/self/pagemap", O_RDONLY);
    if(fd == -1)
    {
        printf("open /proc/self/pagemap error\n");
        return;
    }

    if(lseek(fd, v_offset, SEEK_SET) == -1)
    {
        printf("sleek error\n");
        return;
    }

    if(read(fd, &item, sizeof(uint64_t)) != sizeof(uint64_t))
    {
        printf("read item error\n");
        return;
    }

    if((((uint64_t)1 << 63) & item) == 0)
    {
        printf("page present is 0\n");
        return ;
    }

    uint64_t phy_pageIndex = (((uint64_t)1 << 55) - 1) & item;

    *paddr = (phy_pageIndex * pageSize) + page_offset;
}
~
"mem_addr.c" [dos] 46L, 1084C
```

2. 172.31.234.200 (szu) | 3. 172.31.234.200 (szu) (1)

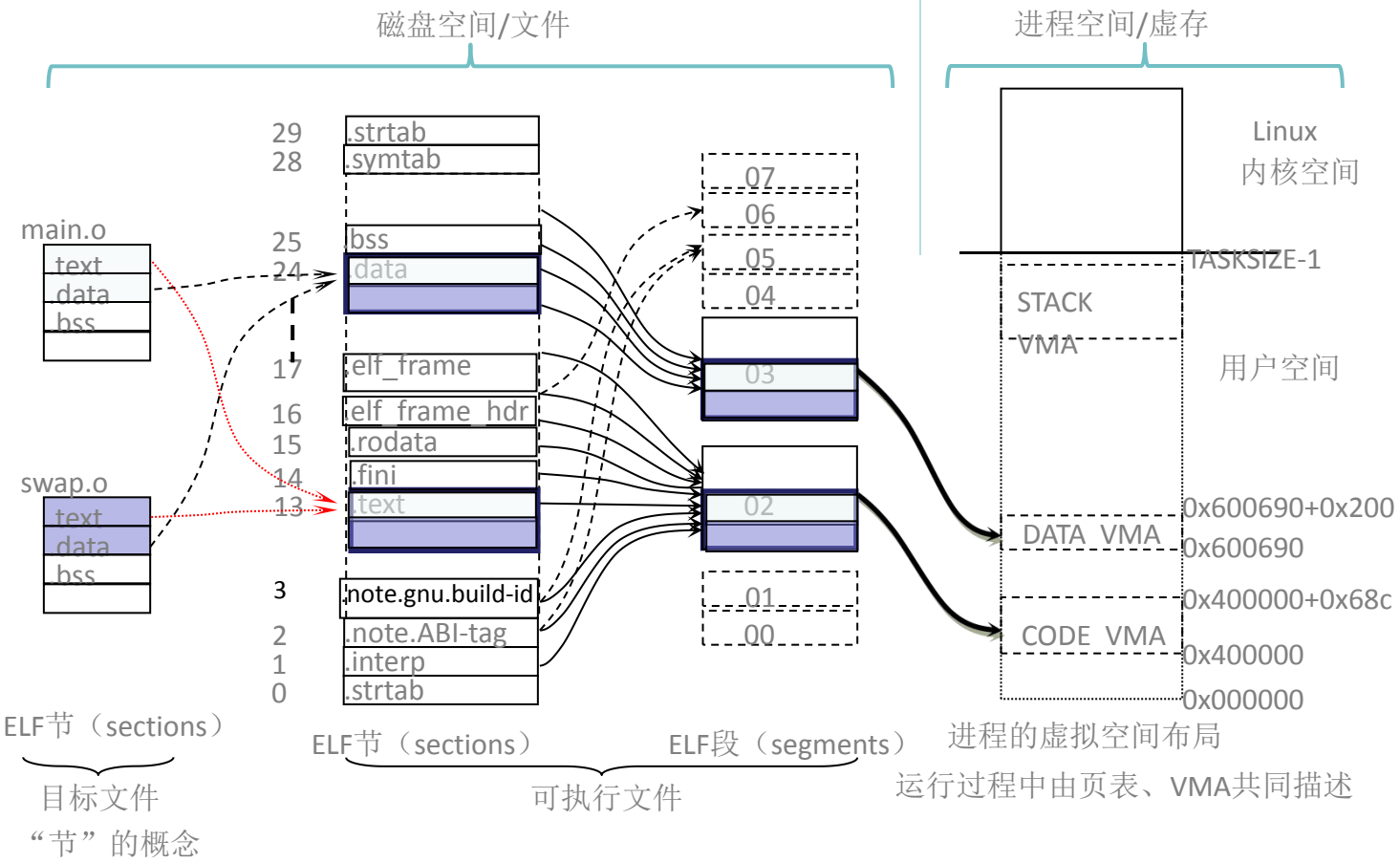
```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>

void mem_addr(unsigned long vaddr,unsigned long *paddr);

int main(int argc, char *argv[])
{
        int i = 5;

        unsigned long phyaddr;

        pid_t pid = fork();

        if (pid < 0){ perror("Fail to fork~\n"); exit(0);}

        if (pid > 0 ) {
                i++;
                unsigned long viraddrPtr = (unsigned long)(&i);
                mem_addr(viraddrPtr,&phyaddr);
                printf("i = %d with virtual addr = %x and physical addr = %x\n", i, &i, phyaddr);
        } else {
                sleep(1);
                unsigned long viraddrPtr = (unsigned long)(&i);
                mem_addr(viraddrPtr,&phyaddr);
                printf("i = %d with virtual addr = %x and physical addr = %x\n", i, &i, phyaddr);
        }
}
~
~
"variables.c" 32L, 739C
```

```
[szu@taishan02-vm-10 process]$ sudo ./variables

我们信任您已经从系统管理员那里了解了日常注意事项。
总结起来无外乎这三点：

    #1) 尊重别人的隐私。
    #2) 输入前要先考虑(后果和风险)。
    #3) 权力越大，责任越大。

[sudo] szu 的密码：
i = 6 with virtual addr = c4f75b44 and physical addr = 3e1c5b44
[szu@taishan02-vm-10 process]$ i = 5 with virtual addr = c4f75b44 and physical addr = 3e
1d5b44
```

https://blog.csdn.net/kongkongkkk/article/details/74366200

《深入理解计算机系统》第四章

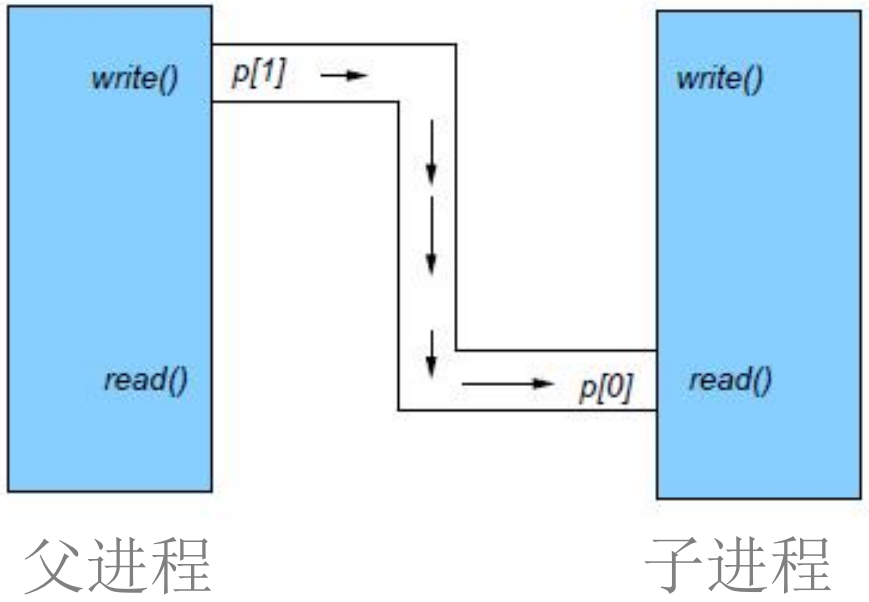
目标文件-可执行文件-进程影像空间的关系



上节课内容

- 进程间通信：what & why
- 管道

```
int pipefd[2];
if (pipe(pipefd) == -1) {
    perror("pipe\n");
    exit(EXIT_FAILURE);
}
```



- 命名管道
- 信号
- 信号量

实验一问题：问（王启飞），详细解答（邓凯誉），答案整理（缪克达）

5. Linux 管道串起的进程组中各进程是从右向左执行还是从左到右执行？如下所示，哪个程序先执行？谁是进程组组长？请设计程序运行并截屏分析。
$a_1.out | a_2.out | a_3.out（20 分）

```
 1 #include <stdio.h>
 2 #include <stdlib.h>
 3 #include <string.h>
 4 #define CMDLEN 20
 5 int main(int argc, char *argv[])
 6 {
 7         char chars;
 8         char cmd[CMDLEN];
 9         memset(cmd,'\0',CMDLEN);
10         while ((chars = getchar()) != '*'){
11                 putchar(chars);
12         }
13         printf("Subjob No. %c\n",*argv[1]);
14         strcpy(cmd, "echo ");
15         cmd[strlen(cmd)]=*argv[1];
16         system(cmd);
17         putchar('*');
18 }
```

启飞同学的困惑

- 任务1-3
  第16行全部倒序输出
  第13行随后顺序输出
- 任务4
  第13行先输出
  第16行再输出

```
[szu@taishan02-vm-10 pipebuf]$ ./pipebuf 1 | ./pipebuf 2 | ./pipebuf 3 | ./pipebuf 4
*
3
2
1
Subjob No. 1
Subjob No. 2
Subjob No. 3
Subjob No. 4
4
*[szu@taishan02-vm-10 pipebuf]$
```

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
        int fd;
        close(1);
        fd = open("text.txt",O_CREAT|O_RDWR,0644);
        if (fd < 0) {
                perror("fopen text.txt error\n");
                exit(0);
        }
        printf("printf Hello World 1\n");
        printf("Hello world 3\n");
        close(fd);
        return 0;
}
```

1. 问题

1.1 文件text.txt为空

1.2 运行后，文件text.txt的内容是?

2. 删除close(fd)

2.1 文件text.txt为空

2.2 运行后，文件text.txt的内容是?

2. 172.31.234.200 (szu)

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
        FILE *fd;
        fd = fopen("text.txt","a");
        if (fd < 0) {
                perror("fopen text.txt error\n");
                exit(0);
        }
        printf("printf Hello World 1\n");
        fprintf(fd,"Hello world 2\n");
        dup2(fileno(fd),1);
        printf("Hello world 3\n");
        fprintf(fd,"Hello world 4\n");
        fclose(fd);
        close(1);
        return 0;
}
```

```
[szu@taishan02-vm-10 fifo]$ ./stdout2file2
printf Hello World 1
[szu@taishan02-vm-10 fifo]$ cat text.txt
Hello world 3
Hello world 2
Hello world 4
```

问题，文件text.txt

- 为何顺序是3、2、4 不是2、3、4
- 为何不为空？

2. 172.31.234.200 (szu)

```
CLOSE(2)                    Linux Programmer's Manual                    CLOSE(2)

NAME
       close - close a file descriptor

SYNOPSIS
       #include <unistd.h>

       int close(int fd);

DESCRIPTION
       close()  closes  a file descriptor, so that it no longer refers to any file and
       may be reused.  Any record locks (see fcntl(2)) held on the file it was associ-
       ated  with,  and  owned by the process, are removed (regardless of the file de-
       scriptor that was used to obtain the lock).

       If fd is the last file descriptor referring to the  underlying  open  file  de-
       scription  (see  open(2)), the resources associated with the open file descrip-
       tion are freed; if the descriptor was the last reference to a  file  which  has
       been removed using unlink(2) the file is deleted.

RETURN VALUE
       close()  returns  zero  on success.  On error, -1 is returned, and errno is set
       appropriately.
```

2. 172.31.234.200 (szu)

```
FCLOSE(3)                   Linux Programmer's Manual                   FCLOSE(3)

NAME
       fclose - close a stream

SYNOPSIS
       #include <stdio.h>

       int fclose(FILE *fp);

DESCRIPTION
       The fclose() function flushes the stream pointed to by fp (writing any buffered
       output data using fflush(3)) and closes the underlying file descriptor.
```

2. 172.31.234.200 (szu)

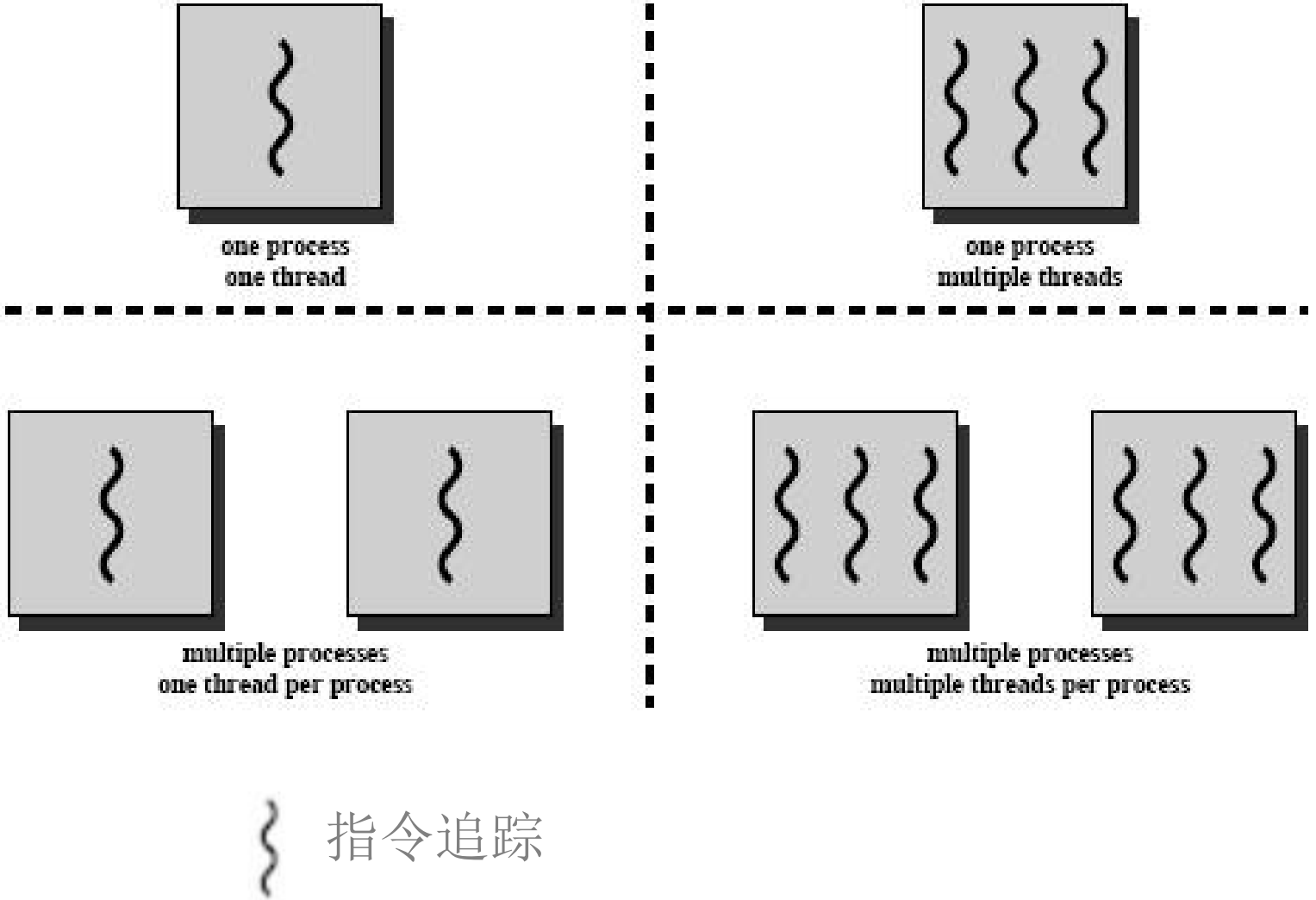
```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
        FILE *fd;
        fd = fopen("text.txt","a");
        if (fd < 0) {
                perror("fopen text.txt error\n");
                exit(0);
        }
        printf("printf Hello World 1\n");
        fprintf(fd,"Hello world 2\n");
        dup2(fileno(fd),1);
        printf("Hello world 3\n");
        fprintf(fd,"Hello world 4\n");
        fclose(fd);
        close(1);
        return 0;
}
```

```
[szu@taishan02-vm-10 fifo]$ ./stdout2file2
printf Hello World 1
[szu@taishan02-vm-10 fifo]$ cat text.txt
Hello world 3
Hello world 2
Hello world 4
```

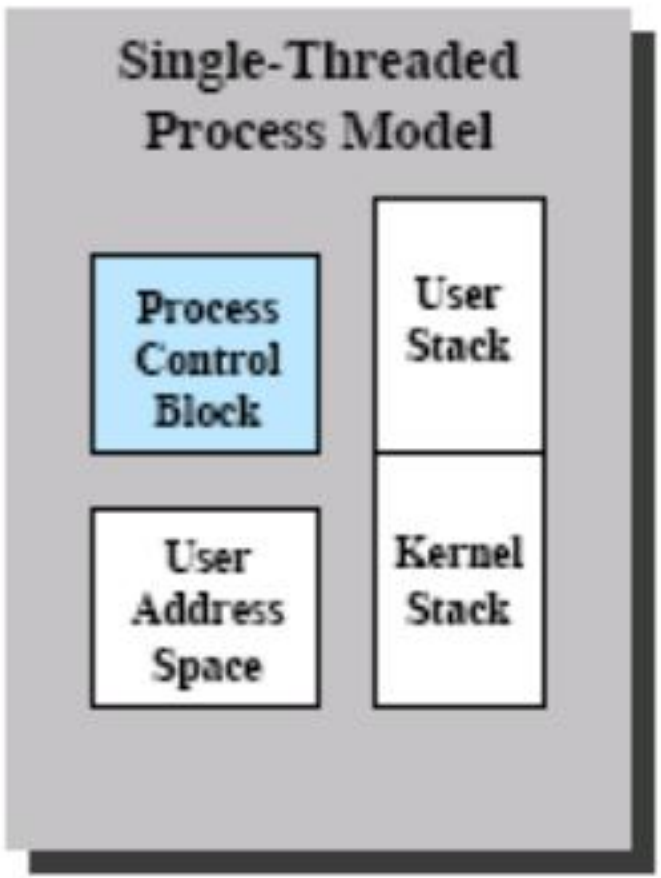
问题，文件text.txt

- 为何顺序是3、2、4 不是2、3、4
- 为何不为空？

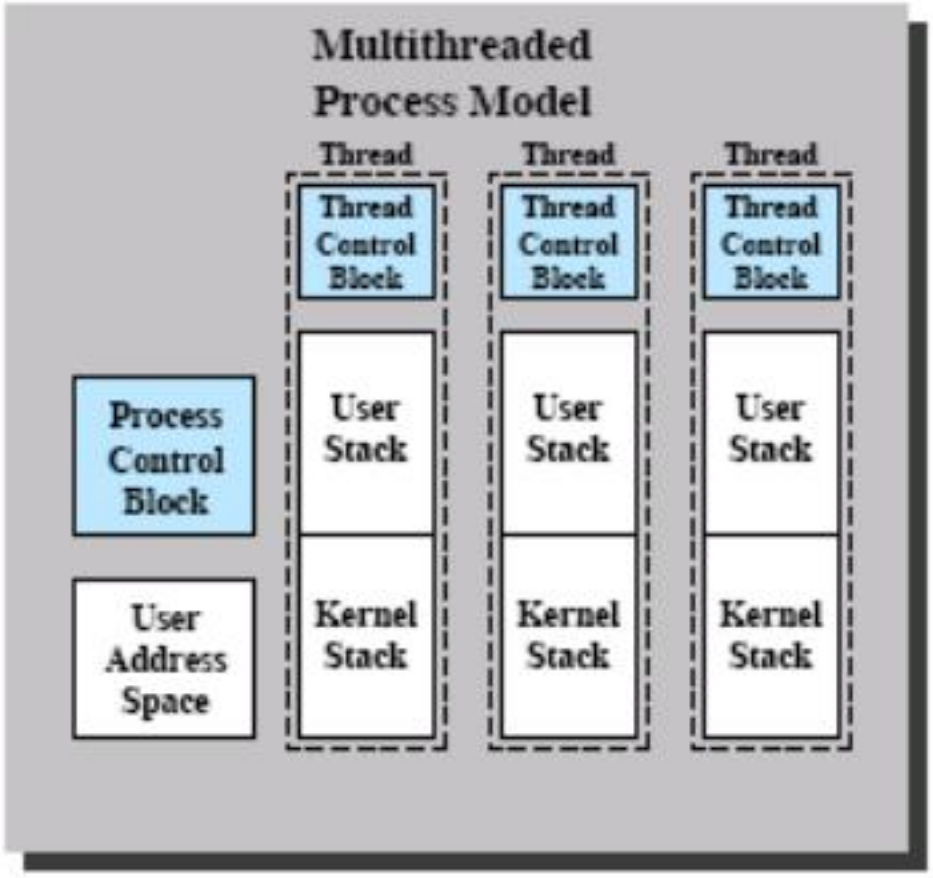
操作系统中的线程和进程



单线程进程模型 vs.多线程进程模型



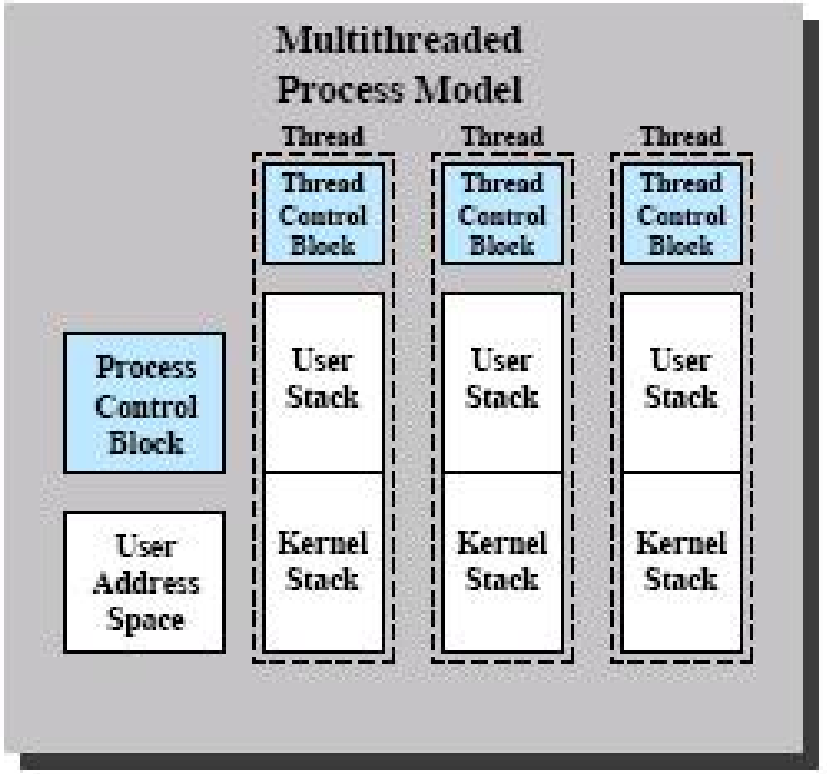
DOS



Solaris, Linux, Window 2000

线程 vs. 进程

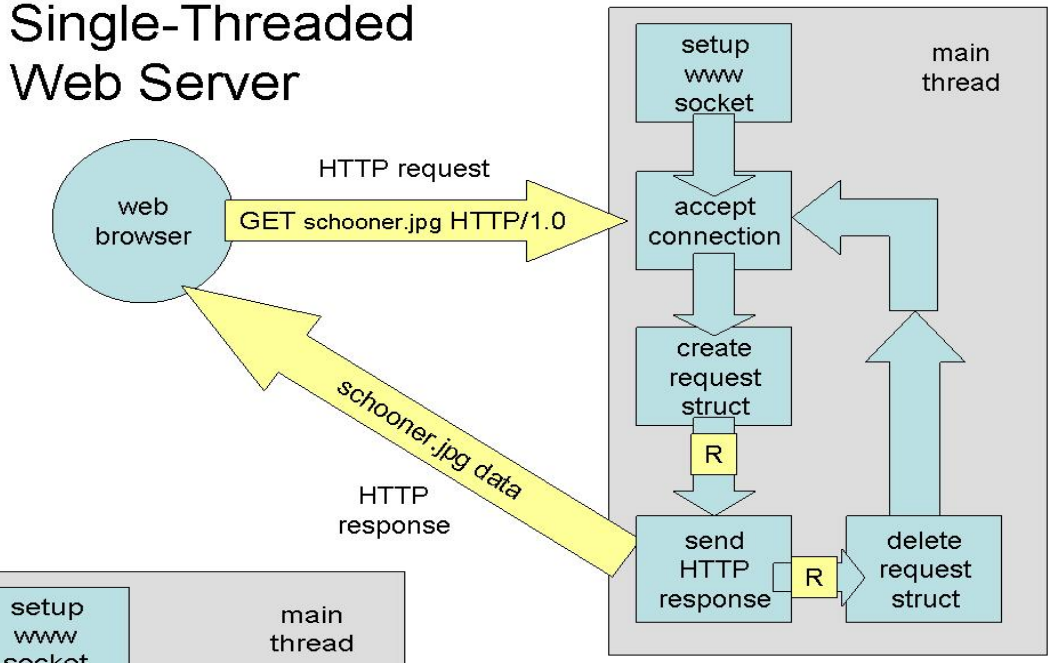
- 生成时间短
- 退出时间短
- 线程间通信时间短
  - 共享地址空间
  - 不用内核传递消息
- 线程间切换时间短
  - 进程资源为其所有线程共享
  - 不用恢复用户地址空间

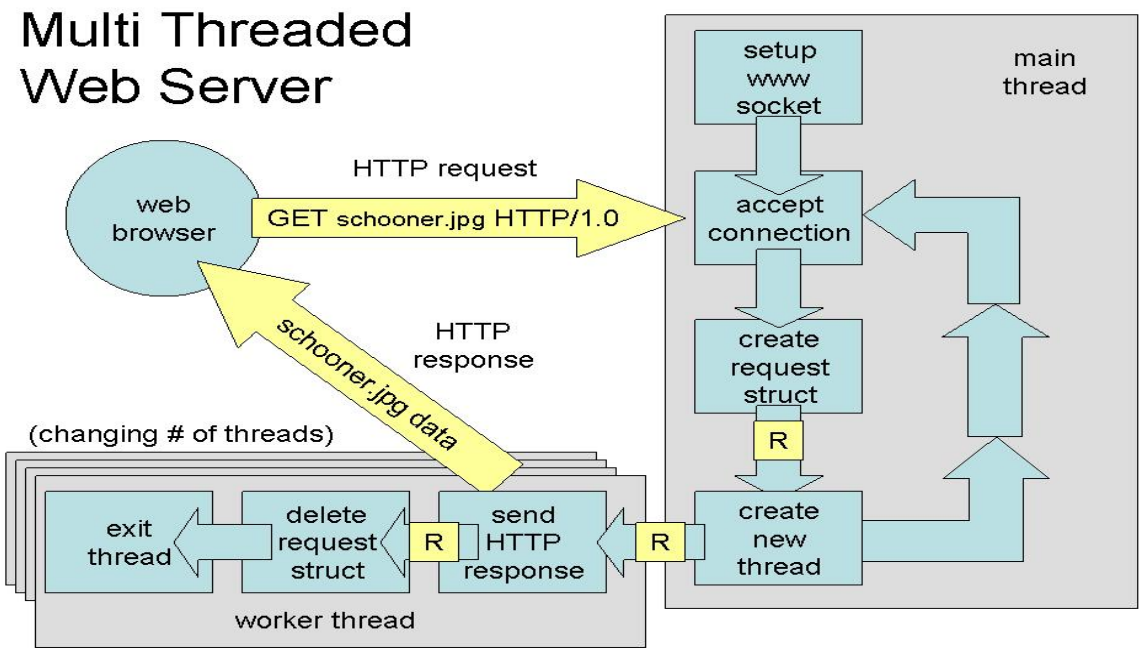


- 进程
  - 资源分配单元
  - 处理机分配单元
- 线程
  - 处理机分配单元

线程的应用例子

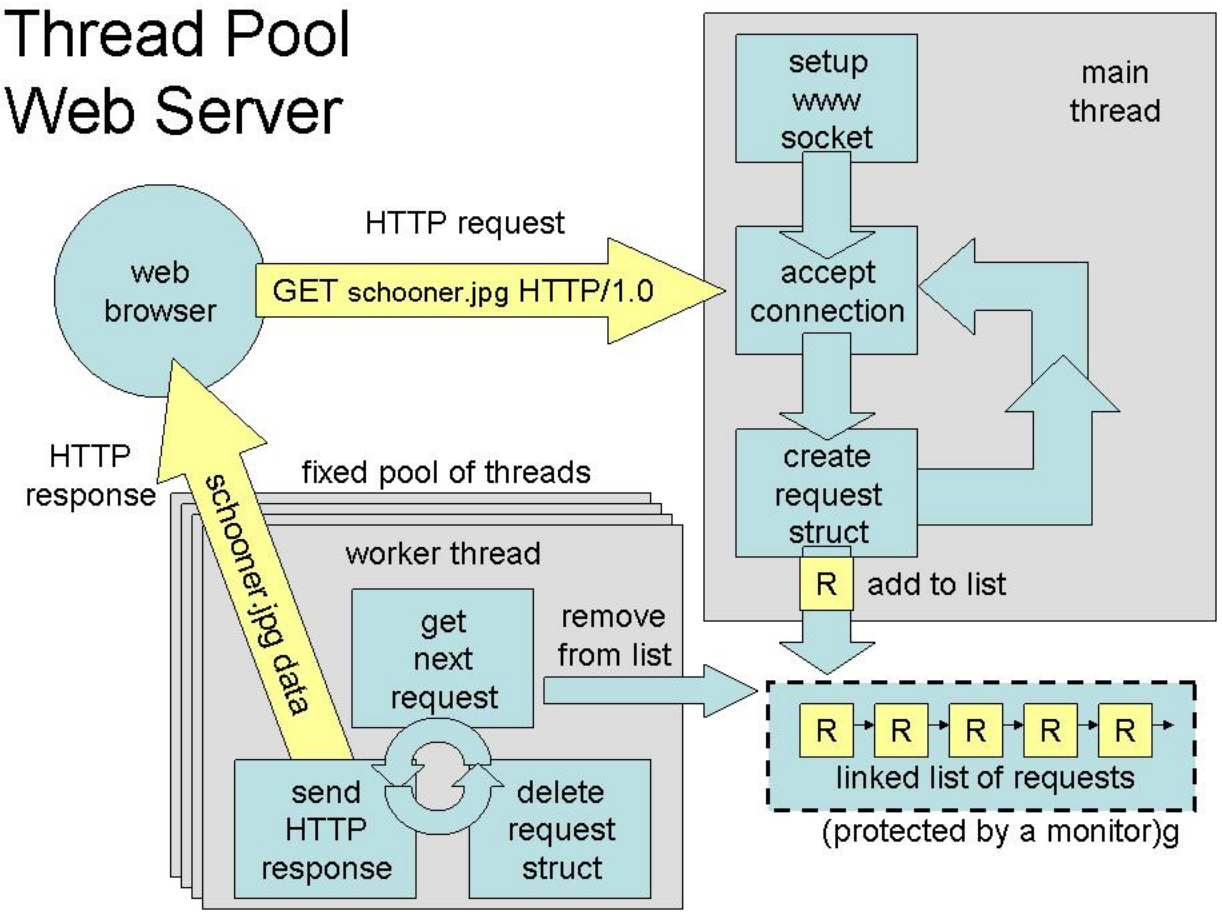
- **Web 服务器**



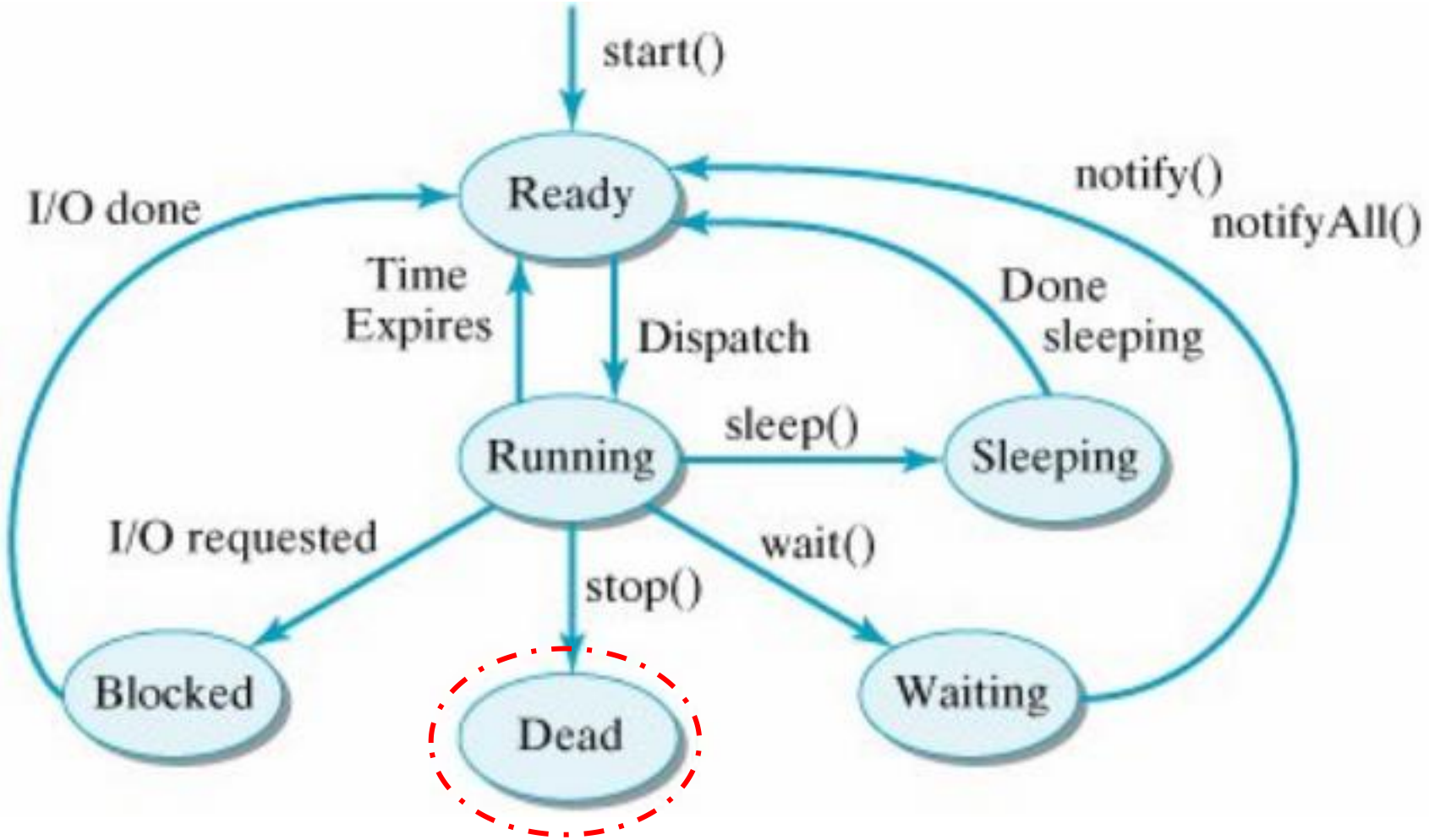


线程的应用例子

- **Web 服务器**



线程状态：一个线程的生命周期

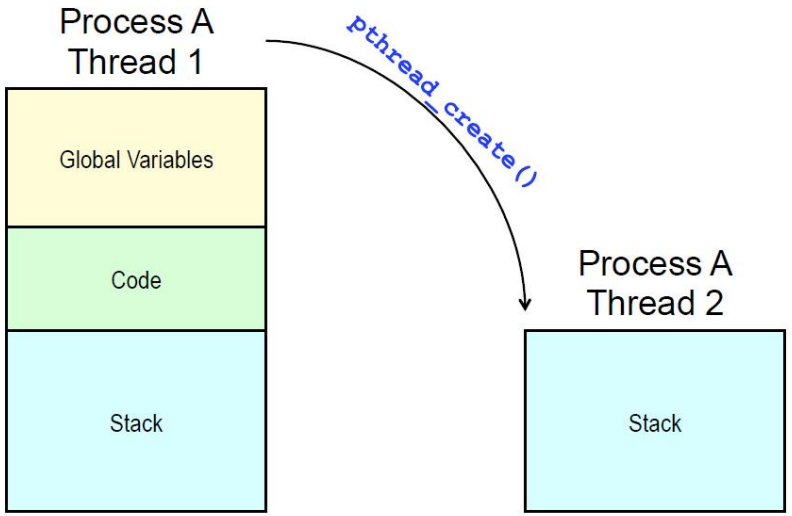


线程控制 – 线程创建

pthread_create()

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);

Compile and link with -pthread.
```

- thread: 指向线程标识符的指针
- attr: 设置线程属性
- start_routine: 线程运行函数的起址
- arg: 运行函数的参数



✓ 没有复制整个进程

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);
```

yuhong@FedoraDVD13:~/C

文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)

```
#include <pthread.h>
#include <stdio.h>

/* Prints x's to stderr, The parameter is unused, does not return. */

void* print_xs(void* unused)
{
        while (1)
                fputc('o',stderr);
        return NULL;
}

/* The main program. */
int main()
{
        pthread_t thread_id;

        pthread_create(&thread_id,NULL,&print_xs,NULL);
        while(1)
                fputc('_',stderr);
        return 0;
}
~
```

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);
```

- void* 型参数：如果需要传输多项类型不同数据
  - 为线程函数定义结构类型,包含其所需的所有数据
  - 多个线程可复用同一线程函数
    - SIMD

2. 172.31.234.200 (szu)

Re-attach Fullscreen Stay on top Duplicate

```
#include <pthread.h>
#include <stdio.h>

struct n_chars {
        char chars;
        int  amount;
};

void *print_n_char(void *params)
{
        struct n_chars *p = (struct n_chars*)params;
        int i;

        for (i = 0; i < p->amount; i++)
                fputc(p->chars,stderr);
        return NULL;
}

int main(int argc, char *argv[])
{
        pthread_t t_id1, t_id2;
        struct n_chars t1_args, t2_args;

        t1_args.chars = '*';
        t1_args.amount = 3;
        pthread_create(&t_id1, NULL, &print_n_char, &t1_args);

        t2_args.chars = '^';
        t2_args.amount = 6;
        pthread_create(&t_id2, NULL, &print_n_char, &t2_args);
        return 0;
}
```

```
[szu@taishan02-vm-10 threads]$ gcc -o join join.c -lpthread
[szu@taishan02-vm-10 threads]$ ./join
[szu@taishan02-vm-10 threads]$ ./join
[szu@taishan02-vm-10 threads]$ ./join
**^^^^^^^[szu@taishan02-vm-10 threads]$ ./join
[szu@taishan02-vm-10 threads]$ ./join
**^^^[szu@taishan02-vm-10 threads]$ ./join
**^^[szu@taishan02-vm-10 threads]$
```

- 创建进程的同时创建主线程
  - 主线程管理进程资源
  - 主线程退出
    - 进程结束
    - 资源回收
    - 其线程退出
- 线程间没有父子关系

线程控制 – 等待指定线程的结束

```
#include <pthread.h>

int pthread_join(pthread_t thread, void **retval);

Compile and link with -pthread.
```

- 参数
  - thread：等待线程的线程ID
  - retval：等待线程函数的返回值
- 返回值
  - 成功：0
  - 出错：error number

出错代码	出错原因
EDEADLK	死锁
EINVAL	● thread线程属性不是joinable ● 有另一个进程等待join线程thread
ESRCH	找不到ID为thread的线程

系统没有定义多个线程同时等待同一个线程的行为

2. 172.31.234.200 (szu)

Re-attach Fullscreen Stay on to

```
#include <pthread.h>
#include <stdio.h>

struct n_chars {
        char chars;
        int  amount;
};

void *print_n_char(void *params)
{
        struct n_chars *p = (struct n_chars*)params;
        int i;

        for (i = 0; i < p->amount; i++)
                fputc(p->chars,stderr);
        return NULL;
}

int main(int argc, char *argv[])
{
        pthread_t t_id1, t_id2;
        struct n_chars t1_args, t2_args;

        t1_args.chars = '*';
        t1_args.amount = 3;
        pthread_create(&t_id1, NULL, &print_n_char, &t1_args);

        t2_args.chars = '^';
        t2_args.amount = 6;
        pthread_create(&t_id2, NULL, &print_n_char, &t2_args);

        pthread_join(t_id1, NULL);
        pthread_join(t_id2, NULL);
        return 0;
}
```

```
[szu@taishan02-vm-10 threads]$ gcc -o withjoin withjoin.c -lpthread
[szu@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^^[szu@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^^[szu@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^^[szu@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^^[szu@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^^[szu@taishan02-vm-10 threads]$
```

- 等待线程结束，释放相应资源
  - 有些线程选择自己释放资源
    - 线程属性为detached
    - 不能被pthread_join()
- 等待joinable线程失败
  - 僵死线程(zombie thread)
  - 消耗系统资源

线程控制 – 获取调用线程的ID

```
#include <pthread.h>
pthread_t pthread_self(void);
```

- 返回值：调用线程的ID，永远成功

线程控制 – 终止线程

- 自主终止
  - 线程函数结束
  - 函数pthread_exit()

```
#include <pthread.h>

void pthread_exit(void *retval);
```

```
int pthread_join(pthread_t thread, void **retval);
```

If retval is not NULL, then pthread_join() copies the exit status of the target thread (i.e., the value that the target thread supplied to pthread_exit(3)) into the location pointed to by *retval. If the target thread was canceled, then PTHREAD_CANCELED is placed in *retval.

```c
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

void* adder(void* arg)
{
        int *operand = malloc(sizeof(int));
        *operand = *((int *)arg);
        *operand = 2 + *operand;

        pthread_exit((void*)operand);

}

int main()
{
        pthread_t thread;
        int *result;
        const int operand=3;
        pthread_create(&thread, NULL, &adder, (void *)&operand);
        pthread_join(thread,(void*)&result);
        printf("The result of the adder is %d.\n",*result);
}
```

获取线程退出状态的例程（一）

```
[szu@taishan02-vm-10 threads]$ gcc -lpthread -o returnvalue returnvalue.c
[szu@taishan02-vm-10 threads]$ ./returnvalue
The result of the adder is 5.
```

```
[szu@taishan02-vm-10 threads]$ gcc -lpthread -o exitstatus exitstatus.c
[szu@taishan02-vm-10 threads]$ ./exitstatus
Thread 1 exits with code 1
Thread 2 exits with code 2
[szu@taishan02-vm-10 threads]$ cat exitstatus.c
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

void *thr_fn1(void *arg)
{
        return ((void *)1);
}

void *thr_fn2(void *arg)
{
        pthread_exit((void *)2);
}


int main()
{
        pthread_t tid1, tid2;
        void *tret1, *tret2;

        if (pthread_create(&tid1, NULL, thr_fn1, NULL) < 0) {
                perror("Fail to create pthread tid1.\n");
                exit(-1);
        }

        if (pthread_create(&tid2, NULL, thr_fn2, NULL) < 0) {
                perror("Fail to create pthread tid2.\n");
                exit(-1);
        }

        pthread_join(tid1,&tret1);
        pthread_join(tid2,&tret2);
        printf("Thread 1 exits with code %d\n",(int *)tret1);
        printf("Thread 2 exits with code %d\n",(int *)tret2);
        exit(0);
}
[szu@taishan02-vm-10 threads]$
```

获取线程退出状态的例程（二）

yuhong@localhost:~/Cprogramming/thread

File Edit View Terminal Help

课堂练习：请将以下代码调试正确

```
void* compute_prime(void* arg)
{
        int candidate = 2;
        int n = *((int*)arg);

        while (1) {
                int factor;
                int is_prime = 1;
                /*Test primality by successive division*/
                for (factor = 2; factor < candidate; ++factor)
                        if (candidate % factor == 0){
                                is_prime = 0;
                                break;
                        }

                /*Is this the prime number we're looking for?*/
                if (is_prime) {
                        if (--n == 0)
                                /*Return the desired prime number as the thread return value. */
                                return (void*) candidate;
                }
                ++candidate;
        }
        return NULL;
}

int main()
{
        pthread_t thread;
        int which_prime = 10;
        int prime;

        pthread_create(&thread,NULL,&compute_prime,&which_prime);
        sleep(10);
        pthread_join(thread,(void*)&prime);
        printf("The %dth prime number is %d.\n", which_prime,prime);
        return 0;
}
```

```c
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

void *compute_prime(void *arg)
{
        int *candidate = NULL;
        int n = *((int *)arg);
        candidate = malloc(sizeof(int));
        *candidate = 2;
        while (1){
                int factor;
                int is_prime = 1;
                for (factor = 2; factor < *candidate; ++factor)
                        if (*candidate % factor == 0){is_prime = 0; break;}
                if (is_prime){
                        if (--n == 0) return candidate;
                }
                ++(* candidate);
        }
        return NULL;
}

int main()
{
        pthread_t tid;
        int which_prime = 10;
        void *prime;
        if (pthread_create(&tid, NULL, &compute_prime, &which_prime) < 0) {
                perror("Fail to create pthread tid1.\n"); exit(EXIT_FAILURE);
        }
        pthread_join(tid,&prime);
        printf("The %dth prime number is %d.\n",which_prime,*((int *)prime));
        exit(EXIT_SUCCESS);
}
```

参考修改

线程控制 – 终止线程

- 被动终止: pthread_cancel()

```
#include <pthread.h>
  int pthread_cancel(pthread_t thread);
```

成功：返回0　　失败：返回非0的出错代码

成功并不意味着thread会终止，与目标线程的两个属性有关

- cancelability state
  - PTHREAD_CANCEL_ENABLE(缺省):取决于cancelability type
    - PTHREAD_CANCEL_ASYNCHRONOUS: 任意时间都可取消该线程
    - PTHREAD_CANCEL_DEFERRED:设置可取消点,收到取消请求，将其放进队列,等线程运行至下一个取消点再退出
  - PTHREAD_CANCEL_DISENABLE:取消请求被阻塞直到状态改变

- pthread_cancel()应用例子
- 在man手册页中

```c
#include <pthread.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>
#define handle_error_en(en, msg) \
        do { errno = en; perror(msg); exit(EXIT_FAILURE); } while (0)

static void * thread_func(void *ignored_argument)
{
        int s;
        if ((s = pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL)) != 0)
                handle_error_en(s, "pthread_setcancelstate");
        printf("thread_func(): started; cancellation disabled\n");
        sleep(5);
        printf("thread_func(): about to enable cancellation\n");
        if ((s = pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL)) != 0)
                handle_error_en(s, "pthread_setcancelstate");
        sleep(1000);        /* Should get canceled while we sleep */
        /* Should never get here */
        printf("thread_func(): not canceled!\n");
        return NULL;
}
```

```c
int main(void)
{
        pthread_t thr;
        void *res;
        int s;
        if ((s = pthread_create(&thr, NULL, &thread_func, NULL)) != 0)
                handle_error_en(s, "pthread_create");
        sleep(2);           /* Give thread a chance to get started */
        printf("main(): sending cancellation request\n");
        if ((s = pthread_cancel(thr)) != 0)
                handle_error_en(s, "pthread_cancel");
        if ((s = pthread_join(thr, &res)) != 0)
                handle_error_en(s, "pthread_join");
        if (res == PTHREAD_CANCELED)
                printf("main(): thread was canceled\n");
        else
                printf("main(): thread wasn't canceled (shouldn't happen!)\n");
        exit(EXIT_SUCCESS);
}
```

```
[szu@taishan02-vm-10 threads]$ gcc -o cancelthread cancelthread.c -lpthread
[szu@taishan02-vm-10 threads]$ ./cancelthread
thread_func(): started; cancellation disabled
main(): sending cancellation request
thread_func(): about to enable cancellation
main(): thread was canceled
```